

AI-Powered Customer Support Triage

Multi-Agent RAG Pipeline with Pinecone & Gemini

Built as a Research PoC | [Github](#)

Nelson Chou

Problem Statement- Customer support operations rely on inconsistent human judgement to triage, escalate, and resolve issues at scale

Operational Frictions

- ✗ Feedback & escalation decisions **vary by agent**
- ✗ Supervisor resolution is **manual and repetitive**
- ✗ **Lack of** institutional memory



Business Impact

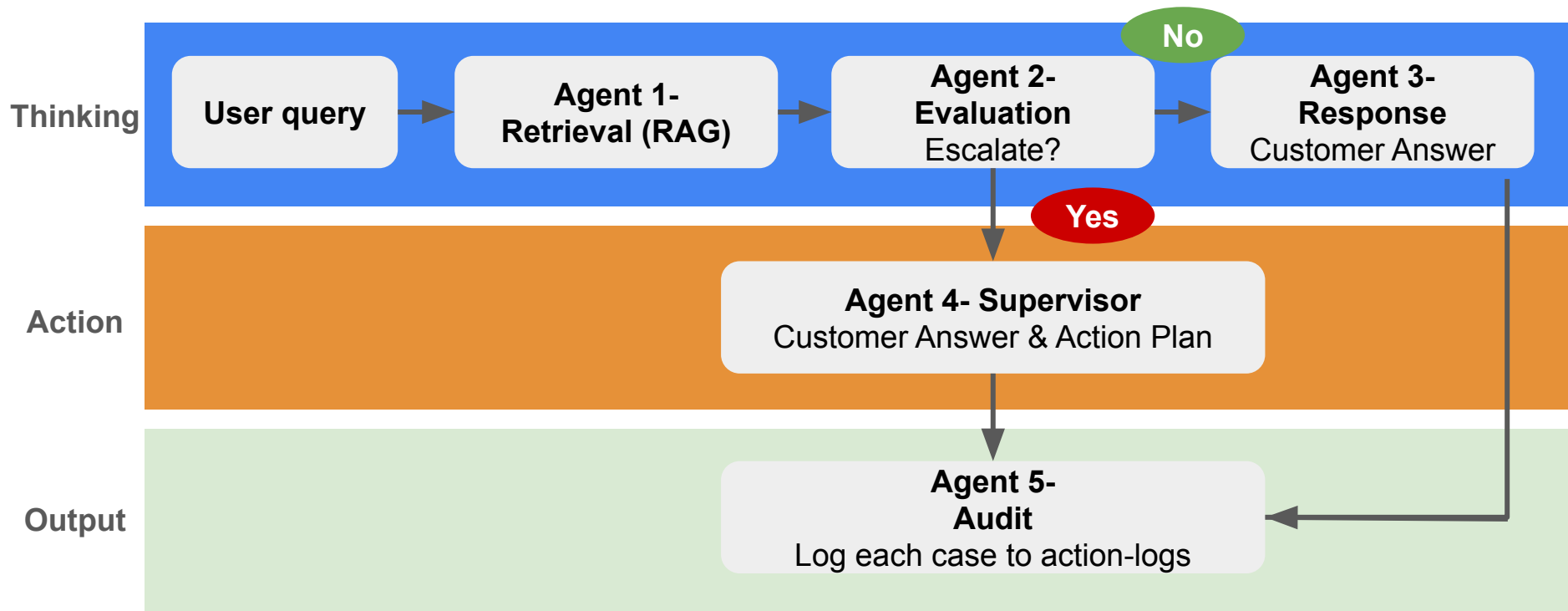
Inconsistent DSAT outcomes

Requires significant human overhead

Same issues handled differently every time

Result: inconsistent customer experience at scale

Solution Overview



An agentic AI pipeline that **standardizes escalation decisions** and **automates supervisor-level resolution recommendations**

Agent 1- Retrieval Agent

INPUT

- User query (raw text)

"I have been waiting for my order for 2 weeks now. The estimated delivery was last Friday and I still haven't received anything. Can someone please look into this?"

PROCESS

- **Embeds** query with text-embedding-004
- **Search** support-docs namespace
- **Search** action-logs namespace (past cases)
- Filter past cases with cosine similarity score > threshold

OUTPUT

- Top N **support docs** (instruction, response, intent)
- Top N **past cases** (query, severity, action taken)
- **Similarity scores** with the user query

Why: Grounds all downstream agents in **real data**, not **LLM memory**

Agent 2- Evaluation Agent

INPUT

- User query
- Support docs
- Past cases

PROCESS

- **Assess** severity against query + context
- **Detect** sentiment
- **Classify** issue
- **Check** repeat issue
- **Decide** escalation (severity score ≥ 4)

OUTPUT

- Severity score (1-5)
- Severity label
- Sentiment
- Issue type
- Repeat issue flag
- Escalation flag
- Suggested tone
- Reasoning

Why: **Standardizes** escalation decisions that previously **varied** by agent

Agent 3- Response Agent

INPUT

- User query
- Support docs
- Severity score
- Sentiment
- Suggested tone

PROCESS

- **Select** tone based on severity and sentiment
- **Ground** answer in retrieved text
- **Generate** response via Gemini

OUTPUT

- Customer facing answer (3-5 sentences)

Why: Ensures response quality is **consistent** regardless of who handles it

1

2

3

4

5

Agent 4- Supervisor Agent

INPUT

- User query
- Evaluation output
- Customer response
- Past cases

PROCESS

- **Only activates** if
Escalation = True
- **Review** all case context
- **Decides** action
- **Recommends**
compensation
- **Assigns** team

OUTPUT

- Action decision
(Compensation / escalate
/ assign / monitor)
- Compensation type
- Assigned team
- Priority level
- Internal notes

Why: **Replicates** centrally governed supervisor logic at scale

1

2

3

4

5

Agent 5- Audit Agent

INPUT

- User query
- Evaluation output
- Customer response
- Supervisor decision

PROCESS

- **Generate** ticket ID
- **Assemble** full interaction records
- **Embed** query vector
- **Upsert** to action-logs namespace

OUTPUT

- **Structured ticket** (logged into Pinecone)
- Ticket ID
- **Searchable past cases** for future retrieval

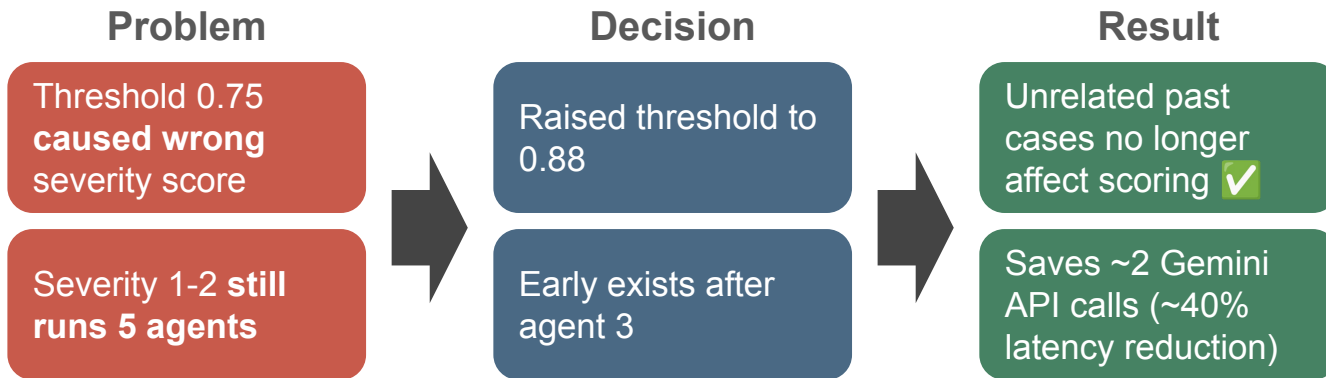
Why: **Builds institutional memory** — every case informs **future decisions**

Key Design Decisions & Trade-offs

Key design decisions

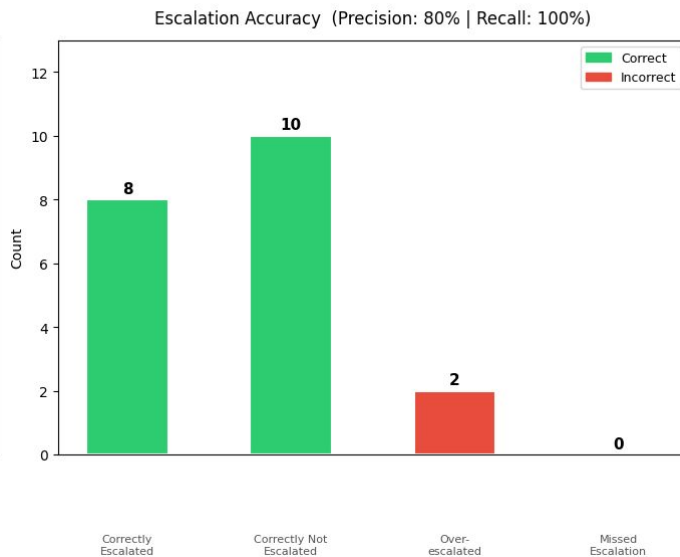
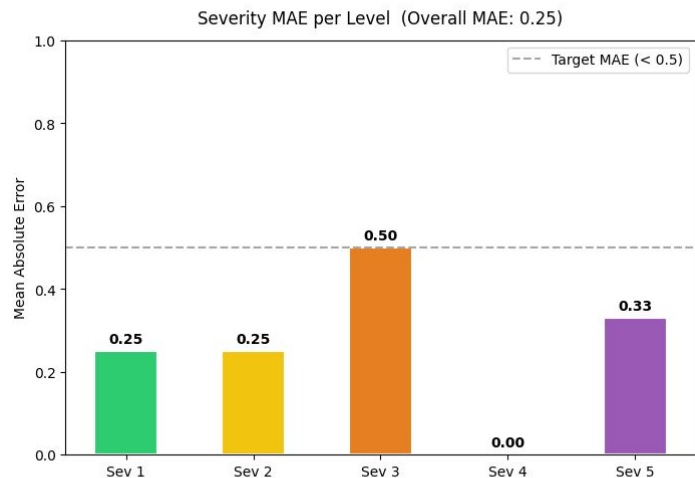
Decision	Option Chosen	Trade-off
What to embed?	Questions, not answers	Misses answer content matches
Severity scoring rules	LLM-based	Non-deterministic across runs
Latency	5 sequential Gemini calls	~30s per query (parallel execution where possible)

Decision in practice



Evaluation Framework & Initial Findings

Multi-Agent RAG Pipeline — Evaluation Results



MAE: 0.25

**Escalation precision:
80%**

The agent pipeline
**performs well on
structured test cases.**

Real-world deployment
would **require
additional stress
testing** on: sarcastic
queries, multilingual
input, and vague
complaints where **intent
is ambiguous.**

Institutional Memory — System Learns from Past Cases

First query

I was charged twice for my order

Past case: 0
Repeat issue: False
Similarity: N/A
Action: None

Second query

Two identical charges on my account

Past case: 1
Repeat issue: Yes
Similarity: 0.82
Action: **Escalate + RCA**

Different words, same meaning - semantic search at work

Production Roadmap - from PoC to Handoff

P0- Must Have Before Handoff

- **PII Redaction:** Strip customer data before embed
- **Error handling:** Per-agent fallback, no cascade fail
- **API service:** FastAPI wrapper for concurrent use

P1- Recommended for Production

- **Ticket integration:** Connect to Zendesk
- **Human Feedback Loop:** Agent ratings improve prompts
- **Monitoring Dashboard:** Escalation rate, latency, drift

P2- Scale Optimization

- **Multilingual Support:** Embed in target language
- **Hybrid Search:** Vector + keyword combined
- **Fine-tuned Evaluator:** Replace LLM scorer with smaller trained classifier